# 3713. Longest Balanced Substring I

Solved

Medium | Topics | Companies | Hint

You are given a string `s` consisting of lowercase English letters.

A **substring** of `s` is called **balanced** if all **distinct** characters in the **substring** appear the **same** number of times.

Return the **length** of the **longest balanced substring** of `s`.

**Example 1:**

> **Input:** `s = "abbac"`
>
> **Output:** 4
>
> **Explanation:**
>
> The longest balanced substring is `"abba"` because both distinct characters `'a'` and `'b'` each appear exactly 2 times.

**Example 2:**

> **Input:** `s = "zzabccy"`
>
> **Output:** 4
>
> **Explanation:**
>
> The longest balanced substring is `"zabc"` because the distinct characters `'z'`, `'a'`, `'b'`, and `'c'` each appear exactly 1 time.

**Example 3:**

> **Input:** `s = "aba"`
>
> **Output:** 2
>
> **Explanation:**
>
> One of the longest balanced substrings is `"ab"` because both distinct characters `'a'` and `'b'` each appear exactly 1 time. Another longest balanced substring is `"ba"`.

**Constraints:**

- `1 <= s.length <= 1000`
- `s` consists of lowercase English letters.

# Python:

```
class Solution:
    def longestBalanced(self, s: str) -> int:
        n = len(s)

        # Transform char -> int
        s = [ord(char) - ord('a') for char in s]

        result = 0
        for l in range(n):
            if n - l <= result:  # Early exit, can't be bigger
                break
```

```
        cnt = [0] * 26  # Counts of every char
        uniq = maxfreq = 0  # Number of uniq chars and maximum frequency
        for r in range(l, n):
            i = s[r]

            uniq += cnt[i] == 0  # There was no this char before => one more uniq
            cnt[i] += 1
            if cnt[i] > maxfreq:  # Update max frequency
                maxfreq = cnt[i]

            # Check if all uniq chars have maxfreq frequency then update the result
            cur = r - l + 1
            if uniq * maxfreq == cur and cur > result:
                result = cur

    return result
```

# JavaScript:

```
/**
 * @param {string} s
 * @return {number}
 */
var longestBalanced = function(s) {
    const n = s.length;
    let ans = 0;
    for (let i = 0; i < n; i++) {
        const freq = new Array(26).fill(0);
        for (let j = i; j < n; j++) {
            freq[s.charCodeAt(j) - 97]++;
            let min = Infinity, max = 0;

            for (const f of freq) {
                if (f > 0) {
                    min = Math.min(min, f);
                    max = Math.max(max, f);
                }
            }

            if (min === max) {
                ans = Math.max(ans, j - i + 1);
            }
        }
    }
    return ans;
```

};

# Java:

```
class Solution {
    public int longestBalanced(String s) {
        int n = s.length();

        // Transform char -> int
        int[] a = new int[n];
        for (int i = 0; i < n; i++)
            a[i] = s.charAt(i) - 'a';

        int result = 0;
        for (int l = 0; l < n; l++) {
            // Early exit, can't be bigger
            if (n - l <= result)
                break;

            int[] cnt = new int[26]; // Counts of every char
            int uniq = 0, maxfreq = 0; // Number of uniq chars and maximum frequency
            for (int r = l; r < n; r++) {
                int i = a[r];

                // There was no this char before => one more uniq
                if (cnt[i] == 0)
                    uniq++;

                cnt[i]++;
                // Update max frequency
                if (cnt[i] > maxfreq)
                    maxfreq = cnt[i];

                // Check if all uniq chars have maxfreq frequency then update the result
                int cur = r - l + 1;
                if (uniq * maxfreq == cur && cur > result)
                    result = cur;
            }
        }
        return result;
    }
}
```